HTTP Server — Question Bank

Know your own project. Answer these without GPT.

This document contains questions across every layer of your HTTP server project — from raw sockets to protocol parsing to concurrency to performance. The goal is simple: if you built it, you should be able to answer these. Use this as a self-test before interviews and as a guide for what to actually understand while building. Questions marked with ★ are high-probability interview questions.

1. Networking Fundamentals

TCP & Sockets

Q1. Why does HTTP use TCP and not UDP? What property of TCP does HTTP depend on?

Q2. ★ What is a socket? What does it represent at the OS level?

Q3. What is the difference between a listening socket and a connected socket? Why do you need both?

Q4. Walk through every syscall your server makes from startup to handling one request. What does each one do?

Q5. What does bind() actually do? What happens if you call bind() on a port already in use?

Q6. What does listen() do and what is the backlog parameter? What happens when the backlog is full?

Q7. What does accept() return? What would happen if you called accept() on a non-listening socket?

Q8. What is the difference between AF_INET and AF_INET6? Can your server handle IPv6?

Q9. What is SO_REUSEADDR and why do almost all server implementations set it?

■ *Hint: try restarting your server quickly after killing it without SO_REUSEADDR.*

Q10. What is a port? Who assigns port numbers, and why can't a regular user bind to port 80?

TCP Internals

Q11. ★ Describe the TCP three-way handshake. Where in this process does accept() return?

Q12. What is the TCP four-way teardown? What does TIME_WAIT mean and why does it exist?

Q13. What is Nagle's algorithm? Could it affect your HTTP server's performance? How would you disable it?

Q14. What is a TCP segment vs an IP packet vs an Ethernet frame? How do they relate?

Q15. What happens when the receiver's buffer is full? What does the sender do?

Q16. What is TCP flow control vs congestion control? Are they the same thing?

2. HTTP/1.1 Protocol

Request Structure

Q17. ★ What does a raw HTTP/1.1 GET request look like at the byte level? Write it out.

Q18. What exactly is \r\n and why does HTTP use it instead of just \n?

Q19. What is the structure of an HTTP request? Name every part and what it contains.

Q20. ★ How does your server know where one HTTP request ends and the next begins?

■ *This is non-trivial. Think about Content-Length, chunked encoding, and connection type.*

Q21. What is the difference between the request line, headers, and body?

Q22. Are HTTP headers case-sensitive? What does the spec say?

Q23. What is the Host header and why is it mandatory in HTTP/1.1 but not HTTP/1.0?

Q24. What HTTP methods exist? Which ones have a body? Which ones are idempotent?

Response Structure

Q25. ★ What does a raw HTTP 200 response look like at the byte level? Write it out.

Q26. What is the difference between a 301 and a 302? Between a 401 and a 403?

Q27. What does the Content-Type header tell the client? What happens if it's wrong?

Q28. What is Content-Length and why does the client need it?

Q29. When would you send a response without a body? What status codes imply no body?

Q30. What does Transfer-Encoding: chunked mean? How does the client know the response is done?

Persistent Connections

Q31. ★ What is HTTP keep-alive / persistent connections? Why does HTTP/1.1 enable it by default?

Q32. What is HTTP pipelining? Does your server support it? What problems does it have?

Q33. How does your server decide when to close a connection?

Q34. What is the Connection: close header and when would you send it?

Q35. What is head-of-line blocking in HTTP/1.1? How does HTTP/2 solve it?

3. Parsing & Request Handling

Parsing Logic

Q36. ★ How did you parse the HTTP request? Walk through the algorithm step by step.

Q37. What can go wrong during parsing? List at least 5 malformed inputs your parser should handle.

Q38. How do you handle a request where the headers arrive in multiple read() calls?

■ *TCP does not guarantee that one send() = one recv(). Your parser must handle partial reads.*

Q39. What is a state machine? Did you use one for parsing? Why is it a natural fit here?

Q40. How do you handle extremely long headers or an abnormally large request body? What's the right response code?

Q41. What happens if a client sends an HTTP/1.0 request to your HTTP/1.1 server?

Q42. How would you handle URL-encoded characters in a path (e.g., %20 for space)?

Request Routing

Q43. How does your server map a URL path to a handler? What data structure did you use?

Q44. What is the difference between a static file server and a dynamic request handler?

Q45. If a client requests /../../etc/passwd, what should your server do? How do you prevent path traversal?

Q46. How do you handle query strings in the URL? How do you parse key=value pairs?

4. Concurrency Model

Design Choices

Q47. ★ How does your server handle multiple concurrent clients? What model did you choose?

Q48. What are the three main concurrency models for a server? Compare them: one thread per connection, thread pool, event loop.

Q49. What is a thread pool? Why is it better than spawning a new thread per request?

Q50. ★ What is the C10K problem? Is your server affected by it?

Q51. What is a non-blocking socket? How is it different from a blocking socket?

Q52. What is select()? What are its limitations? Why do people use epoll instead?

Q53. ★ What is epoll? How does it work? What is the difference between level-triggered and edge-triggered mode?

Q54. If you use threads, how do you protect shared data structures? What could go wrong without synchronization?

Practical Concurrency

Q55. What is a race condition? Give a concrete example that could happen in your server.

Q56. What is a deadlock? How would you avoid it?

Q57. What is a mutex? What is a condition variable? When do you need each?

Q58. If two threads handle requests simultaneously, what data is shared between them? What is per-thread?

Q59. What happens when your thread pool is full and a new connection arrives?

Q60. What is the thundering herd problem? How does accept4() with SOCK_NONBLOCK help?

5. Systems & OS Layer

I/O and Buffers

Q61. ★ What does read() return? What do return values of 0 and -1 mean specifically?

Q62. What is a file descriptor? What is the file descriptor table?

Q63. What is the kernel's socket receive buffer? What happens when it fills up?

Q64. What is the difference between buffered and unbuffered I/O? Which does your server use?

Q65. What is sendfile()? When would it be better than read() + write()?

Q66. What is memory-mapped I/O (mmap)? Could it be useful for serving static files?

Process / Memory

Q67. What happens at the OS level when your server calls accept()? Is it blocking?

Q68. What is a zombie process? Could your server create one if you fork()?

Q69. What is virtual memory? What is the difference between stack and heap in your server process?

Q70. What happens when your server process runs out of file descriptors? What error do you get?

Q71. How would you handle SIGPIPE? What causes it in a server context?

■ *SIGPIPE is sent when you write to a socket whose remote end has been closed.*

Q72. What is the difference between a process and a thread at the kernel level?

6. C++ Implementation

Design & Architecture

Q73. ★ Describe your class structure. What are the main classes and what is each responsible for?

Q74. How did you decide what goes in a class vs a free function?

Q75. Where did you use RAII in your server? Why is it important for socket and file descriptor management?

Q76. How do you ensure a socket is always closed even if an exception is thrown?

Q77. Did you use smart pointers? Where and why?

Q78. What is the Rule of Five? Does any class in your server need to implement it?

C++ Specifics

Q79. Why would you use std::string_view instead of std::string when parsing headers?

Q80. What is undefined behavior? Name one place where it could silently occur in socket code.

Q81. What is the difference between std::thread and a raw pthread? When would you prefer one?

Q82. How do you handle errors from syscalls in C++? Do you use exceptions or error codes?

Q83. What is std::atomic? Where might you use it in a multithreaded server?

Q84. What does volatile mean in C++? Is it useful for multithreading? (Careful — common misconception.)

7. Testing & Debugging

Testing

Q85. ★ How do you test your HTTP server? What tools did you use?

Q86. How would you write a unit test for your HTTP parser without running the actual server?

Q87. What is curl and how do you use it to test specific HTTP behaviors?

Q88. How would you simulate a slow client that sends headers one byte at a time?

Q89. How would you test that your server handles 1000 concurrent connections?

Q90. What is ab (Apache Benchmark) or wrk? What metrics do they give you?

Debugging

Q91. ★ How do you use strace to debug your server? What would you look for?

Q92. How do you use Wireshark or tcpdump to inspect the actual HTTP bytes on the wire?

Q93. Your server works fine under low load but crashes at high concurrency. What do you check first?

Q94. How do you detect a memory leak in your server? What tool would you use?

Q95. What is AddressSanitizer (ASan)? How do you enable it and what does it catch?

Q96. What is a core dump? How do you analyze one with gdb?

8. Performance

Q97. ★ What is the bottleneck in your server? How did you measure it?

Q98. What is latency vs throughput? Which matters more for an HTTP server?

Q99. What is a context switch? Why does it matter for server performance?

Q100. What is cache locality? How could poor memory layout hurt your server's performance?

Q101. What is zero-copy I/O? How does sendfile() achieve it?

Q102. What is connection pooling? Is it relevant to a server or only to clients?

Q103. Your server handles 5k req/sec. What would you need to change to hit 100k req/sec?

Q104. What is HTTP/2 multiplexing and why is it faster than HTTP/1.1 pipelining?

9. Big Picture & Design

Q105. ★ Why did you build this project? What did you learn that you couldn't learn any other way?

Q106. What would you do differently if you started over?

Q107. What features would you add to make this production-grade?

Q108. How is your server different from nginx? What does nginx do that yours doesn't?

Q109. What is TLS/HTTPS? What would it take to add TLS support to your server?

Q110. What is HTTP/3 (QUIC)? Why does it use UDP instead of TCP?

Q111. If your server needed to handle file uploads (POST with large body), what would change?

Q112. What is a reverse proxy? Could your HTTP server be extended to act as one?

Q113. What is rate limiting? How would you implement it in your server?

Q114. What is graceful shutdown? How would you implement it — stop accepting new connections, finish in-flight ones?

10. Bonus — Interview Curveballs

Q115. Write the simplest possible HTTP server in C++ from scratch. No classes, just main(). What's the minimum code?

Q116. I send your server a request with Content-Length: 1000 but only send 500 bytes. What happens?

Q117. Two clients connect simultaneously. Walk me through exactly what your server does, step by step.

Q118. Your server is behind a load balancer. The client IP in the socket is the load balancer's IP. How do you get the real client IP?

Q119. What is the difference between HTTP and WebSocket? Could your server be upgraded to handle WebSocket?

Q120. Explain your server to a 12-year-old. Now explain it to a senior kernel engineer. What changes?

★ = High-probability interview question. | Answer every question in your own words, without referencing GPT output. If you can't answer it, that's a gap to fill — not a reason to skip it.